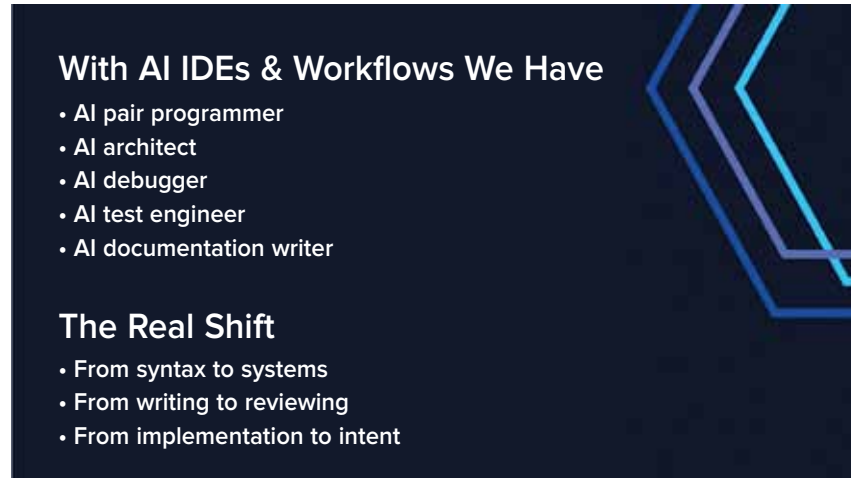
reliable operation. Consequently, AI is increasingly being viewed as a collaborative engineering assistant that supports multiple stages of software development rather than simply generating code snippets.

Within enterprise development environments, AI can assume multiple roles depending on the task. It can function as a pair programmer during implementation, assist architects in evaluating system designs, support debugging activities, act as a test engineer, and simplify technical documentation. This allows developers to concentrate on designing complete systems and solving business problems rather than repetitive engineering tasks. As development practices shift towards intent-driven engineering, organisations increasingly value developers who can translate business objectives into scalable software systems rather than simply produce functional code.

Four jobs, one assistant

AI is increasingly being used across multiple stages of software development. Rather than treating it as another coding assistant, organisations are using it to modernise legacy applications, evaluate architectural choices, connect knowledge systems, and automate repetitive engineering tasks. These applications affect not only how software is built but also how developers approach complex problems.

The code generator. Code generation remains one of the most visible applications of AI in software development, but in enterprise environments, its value extends beyond producing source code to supporting large-scale engineering tasks. Enterprise applications often operate on legacy frameworks that require migration to newer technologies while maintaining compatibility with existing business processes. These migrations demand significant engineering effort, particularly when expertise in older technology stacks becomes increasingly difficult to find.



AI-assisted development workflows supporting multiple engineering roles

One example involved migrating an enterprise application built on an older version of the Vue framework after the framework reached end-of-life. Although developers possessed the technical expertise to perform the migration manually, the migration effort demanded considerable time and specialised knowledge. Traditional approaches relied on external contractors, yet progress remained slow despite months of development effort. After experimenting with multiple AI models and adopting a structured development workflow, the developers successfully completed the migration. The experience demonstrated that successful enterprise AI adoption depends not only on the capabilities of language models but also on providing sufficient context, clear specifications, and well-defined engineering processes.

Successful AI-assisted migration required more than issuing a single prompt. Developers remained responsible for defining application workflows, documenting system behaviour, specifying architectural constraints, and ensuring that generated code aligned with business requirements. Techniques such as specification-driven development and structured documentation helped provide the context required for meaningful outputs, reinforcing the importance of engineering processes alongside model capability.

> Enterprise AI depends as much on well-defined engineering processes as it does on advanced language models.

The thinking partner. Beyond generating code, AI can also support software engineering by helping developers evaluate technical options. Enterprise development frequently involves complex technical constraints that cannot be resolved through generic recommendations. Security requirements, legacy infrastructure, regulatory compliance, and organisational policies often influence architectural decisions, making human judgement essential throughout the design process.

For example, while exploring methods to automate the resolution of vulnerabilities identified through common vulnerabilities and exposures (CVEs), initial AI-generated recommendations focused on standard cloud-based development tools. However, the target environment relied on an on-premises GitHub enterprise server with restricted outbound connectivity, making these recommendations unsuitable. By iteratively providing operational constraints and discussing alternative approaches, developers used AI to evaluate different implementation strategies before identifying a solution that aligned